Proyecto 2

Sistema de Optimización para PFSP (AG, AM y ACS)

Asignatura: Algoritmos Metaheurísticos Inspirados en la Naturaleza
Departamento de Ingeniería Informática — Universidad Católica de la Santísima Concepción (UCSC)

---

1. Contexto y Evolución del Proyecto

El **Proyecto 2** representa la evolución y consolidación del trabajo realizado en la etapa previa:

1. **Proyecto Anterior (PFSP con AG y AM):** Se modeló e implementó el problema de agendamiento industrial *Permutation Flow Shop Scheduling Problem* (PFSP) utilizando dos enfoques evolutivos: **Algoritmos Genéticos (AG)** y **Algoritmos Meméticos (AM)** (AG complementado con búsqueda local).
2. **Preproyecto 2 (Preentrenamiento ACS en TSP):** Se desarrollará y validará un algoritmo de **Sistema de Colonia de Hormigas (Ant Colony System - ACS)** aplicado al Problema del Vendedor Viajero (TSP) **Berlin52** para dominar la inteligencia colectiva, el manejo de feromonas y las reglas de transición.

En esta etapa final, el objetivo es **adaptar e integrar el motor ACS al problema PFSP** y construir un **sistema comparativo unificado** que permita evaluar y contrastar el desempeño de los tres algoritmos (**AG, AM y ACS**) bajo las mismas instancias de prueba y condiciones experimentales.

---

2. Definición Formal del Problema (PFSP)

El **Permutation Flow Shop Scheduling Problem (PFSP)** es un problema de optimización combinatoria en el que un conjunto de trabajos debe procesarse en una serie de máquinas en un orden estricto e idéntico.

Componentes:

- n **Trabajos (Jobs):** $J_1, J_2, \ldots, J_n$
- m **Máquinas:** $M_1, M_2, \ldots, M_m$
- **Secuencia idéntica:** Todos los trabajos pasan secuencialmente por las máquinas en el mismo orden: $M_1 \rightarrow M_2 \rightarrow \cdots \rightarrow M_m$.
- **Matriz de Tiempos de Procesamiento ($p_{i,j}$):** Tiempo fijo que requiere el trabajo i en la máquina j.
- **Restricción de Permutación:** El orden de entrada de los trabajos en la máquina 1 se mantiene inalterado en todas las demás máquinas.

Objetivo y Cálculo del Makespan (C_{max}):

Encontrar la permutación de trabajos $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ que **minimice el Makespan (C_{max})**, correspondiente al tiempo en que el último trabajo finaliza su procesamiento en la última máquina.

Para una secuencia π, la matriz de tiempos de finalización $C(i, j)$ (donde i representa la posición en la secuencia $1 \leq i \leq n$, y j la máquina $1 \leq j \leq m$) se calcula de forma recursiva:

$$C(i,j) = \max\Big(C(i-1,j),\ C(i,j-1)\Big) + p_{\pi_i,j}$$

Con las condiciones de borde $C(0, j) = 0$ y $C(i, 0) = 0$.
El Makespan de la solución es:

$$C_{max}(\pi) = C(n,m)$$

---

3. Algoritmos a Implementar y Comparar

El sistema debe integrar tres metaheurísticas sobre la misma estructura de datos y evaluador de fitness:

3.1. Algoritmo Genético (AG)

- **Representación:** Cromosoma tipo permutación de longitud n.
- **Selección:** Selección por torneo o ruleta.
- **Cruce:** Operador especializado para permutaciones (ej. Order Crossover - OX, o Partially Mapped Crossover - PMX).
- **Mutación:** Operador de mutación por inserción o intercambio (swap).
- **Reemplazo:** Reemplazo generacional con elitismo.

3.2. Algoritmo Memético (AM)

- **Estructura base:** Algoritmo Genético estándar.
- **Búsqueda Local:** Incorporación de una etapa de explotación aplicada a los mejores individuos de la población (o tras un número fijo de generaciones).
- **Operador de Búsqueda Local:** Inserción local (*insertion local search*) o 2-opt adaptado a permutaciones.

3.3. Ant Colony System (ACS)

- **Representación del Grafo:** Grafo de decisión donde los nodos representan los trabajos y las aristas la transición entre programar el trabajo j inmediatamente después del trabajo i.
- **Matriz de Feromona (τ_{ij}):** Indica la deseabilidad aprendida de programar el trabajo j tras el trabajo i.
- **Información Heurística (η_{ij}):** Visibilidad basada en el tiempo de procesamiento (ej. regla de menor tiempo total de procesamiento SPT o heurística NEH reducida).
- **Regla de Transición Pseudo-aleatoria Proporcional:** Elección del siguiente trabajo equilibrando explotación (con probabilidad q_0) y exploración aleatoria proporcional (con probabilidad $1 - q_0$).
- **Actualización Local de Feromona:** Evaporación local en las aristas recorridas para promover la diversidad de rutas en la misma iteración.
- **Actualización Global de Feromona:** Depósito de feromona exclusivo por parte de la mejor hormiga (global o de la iteración) al finalizar la construcción.

---

4. Dataset, Benchmarks y Métricas

Instancias de Prueba:

Se utilizarán las instancias estándar de la literatura de **Taillard**, evaluando al menos tres tamaños de problemas para analizar la escalabilidad:

- **Instancias pequeñas:** $20 \text{ trabajos} \times 5 \text{ máquinas}$.
- **Instancias medianas:** $50 \text{ trabajos} \times 10 \text{ máquinas}$.
- **Instancias grandes:** $100 \text{ trabajos} \times 10 \text{ máquinas}$.

Métrica de Comparación (RPD %):

Dado que cada instancia de Taillard posee un mejor valor conocido (*Upper Bound* / Óptimo), se debe reportar la **Desviación Porcentual Relativa (RPD)**:

$$\text{RPD}(\%) = \frac{C_{max}^{\text{obtenido}} - C_{max}^{\text{óptimo}}}{C_{max}^{\text{óptimo}}} \times 100$$

---

5. Arquitectura del Software

El desarrollo debe priorizar soluciones modulares, claras y mantenibles por sobre arquitecturas complejas o con hiper-ingeniería.

Estructura Recomendada de Archivos:

```
proyecto2_pfsp/
│
├── main.py            # Orquestador principal e interfaz de línea de comandos
├── operadores.py      # Biblioteca modular de funciones (cálculo C_max, AG, AM, ACS)
├── datos/             # Archivos de instancias de Taillard (.txt)
├── resultados/        # Reportes estructurados de salidas (.csv)
├── graficos/          # Gráficos de convergencia (.pdf / .png)
└── README.md          # Instrucciones claras de compilación y ejecución
```

Ejecución por Línea de Comandos:

El programa debe permitir la sintonización de parámetros de forma explícita. Cada algoritmo se ejecutará mediante su propio script ejecutable. Los parámetros se pasan directamente por línea de comandos en un orden fijo y claro. Esto permite crear scripts simples en Bash o Python para lanzar experimentos masivos sobre cada algoritmo sin riesgo de interferencia entre parámetros.

1. Algoritmo Genético (`ag.py`)

Parámetros específicos: *Instancia, Semilla, Población, Iteraciones, Prob. Cruce (p_c), Prob. Mutación (p_m), Archivo Salida*.

```
python ag.py datos/tai20_5.txt 12345 50 200 0.8 0.1 resultados/res_ag.csv
```

2. Algoritmo Memético (`am.py`)

Parámetros específicos: *Instancia, Semilla, Población, Iteraciones, Prob. Cruce (p_c), Prob. Mutación (p_m), Frecuencia/Intensidad Búsqueda Local, Archivo Salida*.

```
python am.py datos/tai20_5.txt 12345 50 200 0.8 0.1 10 resultados/res_am.csv
```

3. Ant Colony System (`acs.py`)

Parámetros específicos: *Instancia, Semilla, N° Hormigas, Iteraciones, Evaporación (α), Peso Heurística (β), Prob. Límite (q_0), Archivo Salida*.

```
python acs.py datos/tai20_5.txt 12345 50 200 1.0 2.0 0.9 resultados/res_acs.csv
```

Reproducibilidad Obligatoria:

El sistema debe garantizar reproducibilidad absoluta mediante el control explícito de la **semilla aleatoria (`seed`)**. Ejecutar el mismo algoritmo con la misma semilla e instancia debe entregar el mismo Makespan y secuencia final.

---

6. Entregables e Informe Científico

1. Código Fuente (50% de la nota final)

- Implementación modular en **Python 3**.
- Código original, documentado y sin dependencias innecesarias.
- Interfaz por pantalla ordenada, con manejo de errores básico.

2. Informe Científico (50% de la nota final)

Elaboración de un artículo científico escrito en inglés o español utilizando la **plantilla IEEE Transactions** (extensión sugerida: 2000 a 3000 palabras).

Estructura del Artículo:

1. **Título, Autores y Afiliación.**
2. **Resumen (Abstract):** Síntesis del problema, metodología comparativa y hallazgos principales.
3. **Introducción:** Contexto del scheduling industrial, definición del PFSP, breve estado del arte, objetivos del trabajo.
4. **Metodología:** Explicación formal de AG, AM y ACS adaptado a PFSP, detallando representaciones, operadores y parámetros.
5. **Resultados Experimentales:**
   - Tablas comparativas de **Makespan (C_{max})** y **RPD (%)**.
   - Comparativa de **Tiempo de CPU** (segundos).
   - Gráficos de las **curvas de convergencia** por iteración.
   - Análisis crítico del comportamiento de cada algoritmo (Exploración vs. Explotación).
6. **Conclusiones y Trabajo Futuro.**
7. **Referencias:** Formato IEEE.

---

7. Pauta de Evaluación

Criterio	Descripción	Porcentaje
Interfaz y Usabilidad	Presentación por consola, manejo de argumentos por CLI, mensajes claros y manejo de errores.	**20%**
Calidad de Código y Modularidad	Uso del principio KISS, modularidad en `operadores.py`, estructura clara de datos y originalidad.	**30%**
Funcionalidad y Resultados	Correcta implementación de AG, AM y ACS, cálculo exacto del C_{max}, reproducibilidad por semillas y calidad del informe IEEE.	**50%**

> **Nota sobre Honestidad Académica:** El trabajo debe ser 100% original. Cualquier evidencia de plagio o copia resultará en evaluación con nota mínima **NCR** conforme al reglamento institucional.